

A NOTE ON

ARTIFICIAL INTELLIGENCE

Somnath Mukherjee

Assistant Professor
MCA, RERF

August 21, 2026

Contents

1	Introduction to Artificial Intelligence	5
1.1	Introduction	5
1.2	What is Artificial Intelligence?	5
1.2.1	What is AI?	5
1.2.2	Current Capabilities of Artificial Intelligence	6
1.2.3	AI vs Conventional Programming	7
1.3	History of Artificial Intelligence	7
1.3.1	Theoretical Foundations (1920s–1930s)	7
1.3.2	The Birth of AI (1950s)	8
1.3.3	Early Growth and Knowledge-Based Systems (1960s–1970s)	8
1.3.4	Neural Networks, Expert Systems, and Renewed Progress (1980s–1990s)	9
1.3.5	The Deep Learning Era (2010s)	9
1.3.6	The Generative AI and Agentic Era (2022–2026)	10
1.4	Definitions of Artificial Intelligence	11
1.4.1	John McCarthy (1956)	11
1.4.2	Elaine Rich (1983)	11
1.4.3	Russell & Norvig (2021)	11
1.5	Approaches to Artificial Intelligence	11
1.5.1	Cognitive Science Approach	11
1.5.2	Turing Test Approach	12
1.5.3	Weak AI (Narrow AI)	12
1.5.4	Strong AI (Artificial General Intelligence)	13
1.5.5	Super AI	13
1.5.6	Applied AI	13
1.6	Problems of Artificial Intelligence	13
1.6.1	Knowledge Representation	14
1.6.2	Reasoning	14
1.6.3	Learning	15
1.6.4	Planning	15
1.6.5	Natural Language Understanding	15
1.6.6	Computer Vision	15
1.6.7	Robotics	15
1.7	AI Techniques	16

1.7.1	Search	16
1.7.2	Knowledge Representation	16
1.7.3	Inference	17
1.7.4	Learning	17
1.7.5	Planning	17
1.8	Tic-Tac-Toe Problem	17
1.8.1	Introduction	17
1.8.2	State Space	17
1.8.3	Game Tree (Basic Idea)	18
1.8.4	Why it is an AI Problem	18
1.9	Chapter Summary	19
1.10	Key Points to Remember	20
1.10.1	Short Questions	20
1.10.2	Long Questions	20
2	Problem Space and Search	21
3	Heuristic Search Techniques	22
4	Game Playing	23
4.1	Introduction	23
4.2	Game Playing as a Search Problem	24
4.2.1	Why Games are Hard for AI	24
4.2.2	Game Trees	24
4.2.3	Two-Player, Zero-Sum Games	25
4.3	The Minimax Search Procedure	25
4.3.1	The Minimax Principle	25
4.3.2	Minimax Algorithm	26
4.3.3	Worked Example	26
4.3.4	Properties of Minimax	27
4.4	Alpha-Beta Pruning	28
4.4.1	Motivation — Reducing the Search Space	28
4.4.2	Alpha and Beta Values	28
4.4.3	Alpha-Beta Algorithm	29
4.4.4	Worked Example	29
4.4.5	Effectiveness of Alpha-Beta Pruning	30
4.5	Iterative Deepening in Game Playing	30
4.5.1	Why Iterative Deepening is Used in Games	30
4.5.2	Iterative Deepening with Alpha-Beta	31
4.6	Additional Refinements	32
4.6.1	Evaluation Functions	32
4.6.2	Horizon Effect	32
4.6.3	Move Ordering	32
4.7	Case Study — Tic-Tac-Toe Revisited	32
4.8	Chapter Summary	33

4.9	Key Points to Remember	34
4.9.1	Short Questions	34
4.9.2	Long Questions	34
5	Knowledge Representation Using Rules	35
6	Slot and Filler Structures	36
7	Logic Programming with Prolog and LISP	37
8	Learning and Machine Learning	38
9	Neural Networks and Deep Learning	39
10	Natural Language Processing	40
11	Expert Systems	41
A	Prolog Programs	42
B	LISP Programs	43
C	University Question Bank	44

Chapter 1

Introduction to Artificial Intelligence

1.1 Introduction

Artificial Intelligence is concerned with the design of intelligence in an artificial device.

The term was coined by McCarthy in 1956.

There are two ideas in the definition.

1. Intelligence
2. Artificial device

Given this scenario different interpretations have been used by different researchers as defining the scope and view of Artificial Intelligence.

1.2 What is Artificial Intelligence?

1.2.1 What is AI?

- There are two possibilities:
 - A system with intelligence is expected to behave as intelligently as a human.
 - A system with intelligence is expected to behave in the best possible manner.
- What type of behavior we are concerned about?
 - Are we looking at the thought process or reasoning ability of the system?
 - Or are we only interested in the final manifestations of the system in terms of its actions?

1.2.2 Current Capabilities of Artificial Intelligence

Modern AI systems have achieved remarkable success in a wide range of specialized domains. With recent advances in Machine Learning, Deep Learning, and Generative AI, intelligent systems are capable of performing tasks that were once considered exclusive to humans. However, current AI systems still lack true human-level intelligence, common-sense reasoning, emotional understanding, and the ability to perform effectively in all situations. Therefore, today's AI is often referred to as *Narrow AI* or *Weak AI*.

The current capabilities of Artificial Intelligence include:

- **Computer Vision:** AI systems can recognize faces, detect and classify objects, analyze medical images, and assist autonomous vehicles in understanding their surroundings.
- **Robotics:** AI enables autonomous robots for manufacturing, warehouse automation, surgical assistance, space exploration, and self-driving vehicles.
- **Natural Language Processing (NLP):** AI systems can perform machine translation, text summarization, sentiment analysis, question answering, document generation, and conversational interactions through intelligent chatbots and virtual assistants.
- **Expert Systems:** AI-based expert systems assist in medical diagnosis, financial planning, fault diagnosis, legal consultation, and decision support within specialized domains.
- **Speech Processing:** AI systems can recognize spoken language, convert speech to text, generate natural-sounding speech, and provide real-time language translation.
- **Planning and Scheduling:** AI is widely used in airline scheduling, logistics, supply chain management, manufacturing, satellite mission planning, and smart transportation systems.
- **Machine Learning:** AI systems can classify images, detect fraud, recommend products, predict customer behavior, and analyze large volumes of data for decision making.
- **Games:** AI systems have achieved superhuman performance in games such as Chess, Go, Poker, Dota 2, and StarCraft II by learning optimal strategies.
- **Generative AI:** Modern AI systems can generate human-like text, images, audio, videos, computer programs, and creative content based on user instructions.
- **Healthcare:** AI assists in disease diagnosis, drug discovery, personalized medicine, medical image analysis, and patient monitoring.
- **Education:** AI supports intelligent tutoring systems, personalized learning, automated assessment, and virtual teaching assistants.

- **Finance and Cybersecurity:** AI is used for fraud detection, credit risk assessment, algorithmic trading, malware detection, intrusion detection, and cybersecurity monitoring.

1.2.3 AI vs Conventional Programming

Artificial Intelligence and Conventional Programming differ in their approach to solving problems. Conventional programming follows explicitly defined rules and algorithms written by the programmer, whereas AI systems learn from data, reason, and make decisions even in uncertain situations.

The major differences between Artificial Intelligence and Conventional Programming are shown in Table 1.1.

Table 1.1: Comparison between Artificial Intelligence and Conventional Programming

Feature	Artificial Intelligence	Conventional Programming
Approach	Learns from data and experience	Follows predefined rules and algorithms
Problem Solving	Can solve complex and uncertain problems	Best suited for well-defined problems
Learning Ability	Can improve through learning and training	Cannot learn unless reprogrammed
Decision Making	Makes intelligent decisions based on knowledge and inference	Makes decisions strictly according to programmed logic
Data Handling	Can process structured and unstructured data	Primarily processes structured data
Adaptability	Adapts to new situations	Limited adaptability
Accuracy	May improve with training but can make probabilistic decisions	Produces deterministic and predictable results
Examples	ChatGPT, Self-driving cars, Face recognition, Recommendation systems	Calculator, Payroll system, Library management system, Banking software

1.3 History of Artificial Intelligence

The development of Artificial Intelligence did not begin with computers themselves, but with foundational questions in mathematics and logic about what could, in principle, be computed or proven. This section traces the major milestones that shaped AI from its theoretical origins to the present day.

1.3.1 Theoretical Foundations (1920s–1930s)

- **1931 — Kurt Gödel** laid the foundations of theoretical computer science by publishing the first universal formal language and proving his *incompleteness theorems* —

showing that any sufficiently powerful formal mathematical system is either inconsistent (flawed) or contains true statements that cannot be proven within the system.

- **1936** — **Alan Turing** reformulated Gödel’s result (along with Alonzo Church’s related extension) in terms of an abstract computing device — now known as the **Turing Machine** — providing a formal definition of what it means for a function to be computable, and laying the mathematical groundwork for the modern computer.

1.3.2 The Birth of AI (1950s)

- **1950** — **Alan Turing** published “*Computing Machinery and Intelligence*,” proposing the **Imitation Game** — later known as the **Turing Test** (Chapter 1, Section 4.2) — as an operational test for machine intelligence.
- **1956** — **John McCarthy** coined the term “**Artificial Intelligence**” as the subject of the **Dartmouth Conference**, the first conference devoted entirely to the field. McCarthy is regarded as the **Father of Artificial Intelligence**.
- **1957** — **Newell, Shaw & Simon** demonstrated the **General Problem Solver (GPS)**, an early attempt at a universal, symbolic problem-solving program (see Chapter 2).
- **1958** — **John McCarthy** (MIT) invented the **Lisp** programming language, which became the dominant language for symbolic AI programming for decades.
- **1959** — **Arthur Samuel** (IBM) wrote the first game-playing program, for Checkers, sufficiently skilled to challenge strong human players — an early demonstration of machine learning through self-play.

1.3.3 Early Growth and Knowledge-Based Systems (1960s–1970s)

- **1963** — **Ivan Sutherland**’s MIT dissertation on *Sketchpad* introduced interactive computer graphics.
- **1966** — **Ross Quillian** (PhD dissertation, Carnegie Institute of Technology, now CMU) demonstrated **semantic networks** as a technique for knowledge representation.
- **1967** — **DENDRAL**, developed by Edward Feigenbaum, Joshua Lederberg, Bruce Buchanan, and Georgia Sutherland at Stanford, interpreted mass spectra of organic chemical compounds — the first successful **knowledge-based** program for scientific reasoning (see Chapter 3).
- **1967** — **Doug Engelbart** invented the computer mouse at SRI.
- **1968** — **Marvin Minsky & Seymour Papert** published *Perceptrons*, mathematically demonstrating the limitations of simple (single-layer) neural networks — a result

that contributed to reduced funding for neural network research over the following decade.

- **1972** — **Alain Colmerauer** developed **Prolog**, the logic programming language central to Unit 3 of this course.

1.3.4 Neural Networks, Expert Systems, and Renewed Progress (1980s–1990s)

- **Mid-1980s** — Neural networks became widely used again following the popularization of the **Backpropagation** algorithm (first described by Werbos in 1974), enabling practical training of multi-layer networks and reviving interest after the setback caused by *Perceptrons*.
- **1990** — Major advances occurred across nearly all areas of AI, with significant demonstrations in machine learning, intelligent tutoring systems, case-based reasoning, multi-agent planning and scheduling, reasoning under uncertainty, data mining, natural language understanding and translation, computer vision, virtual reality, and game-playing.
- **1997** — **Deep Blue** (IBM) defeated World Chess Champion Garry Kasparov — a landmark demonstration of the game-playing and search techniques covered in Chapter 4.
- **2002** — iRobot, founded by researchers from the MIT Artificial Intelligence Lab, introduced the **Roomba** autonomous vacuum-cleaning robot; over two million units had been sold by 2006.

1.3.5 The Deep Learning Era (2010s)

- **2012** — **AlexNet** won the ImageNet competition by a dramatic margin, demonstrating the power of deep convolutional neural networks and triggering the modern deep learning boom.
- **2016** — **AlphaGo** (Google DeepMind), combining deep neural networks with search techniques related to those in Chapter 4, defeated world champion Go player Lee Sedol in Seoul — a milestone widely regarded as far ahead of expectations for that era, since Go’s search space is vastly larger than Chess’s.
- **2017** — Google researchers published “*Attention Is All You Need*,” introducing the **Transformer** architecture, which underpins essentially all modern large language models.
- **2020** — **AlphaFold 2** (Google DeepMind) solved the fifty-year-old **protein-folding problem** at the CASP14 competition, demonstrating AI’s ability to tackle major open

problems in the natural sciences. In the same year, OpenAI released **GPT-3**, a 175-billion-parameter language model showing that scale alone could produce remarkably capable, few-shot language understanding.

1.3.6 The Generative AI and Agentic Era (2022–2026)

- **2022** — **ChatGPT** (OpenAI), based on the GPT-3.5/GPT-4 family, was released as a public research preview and reached 100 million users within two months — the fastest-growing consumer application in history at the time, and the moment generative AI entered mainstream public awareness.
- **2023** — **GPT-4** (OpenAI) demonstrated strong multimodal reasoning; **Anthropic** released **Claude 1**; **Google** released **Gemini 1.0**. Growing concern about the pace of AI progress led many prominent researchers and technologists to sign public statements calling for proactive AI safety governance. The UK hosted the first **AI Safety Summit** at Bletchley Park, the EU advanced the **AI Act**, and the US issued an Executive Order on AI safety — marking the beginning of formal AI regulation at a global scale.
- **2024** — Models such as **GPT-4o**, **Claude 3.5 Sonnet**, and **Gemini 1.5** introduced natively multimodal processing (text, image, audio, and video together). **OpenAI's o1** introduced explicit *chain-of-thought* reasoning at inference time, shifting focus from pure parameter scaling toward reasoning-time computation. The **2024 Nobel Prize in Physics** was awarded to Hopfield and Hinton for foundational work on neural networks, and the **Nobel Prize in Chemistry** was awarded to Hassabis, Jumper, and Baker for AlphaFold's contributions to protein structure prediction — a rare instance of AI research directly earning the field's highest scientific honours.
- **2025** — Often described as the beginning of the "**Age of Agentic AI**": large language models gained the ability to autonomously use tools, browse the web, and execute multi-step coding and research workflows with limited human supervision, rather than simply answering single-turn queries.
- **2026** — AI reached deep integration into everyday industry workflows, with autonomous **AI agents** increasingly handling complex, multi-step tasks (research, coding, and operational workflows) with limited supervision, alongside continued progress in physical/embodied AI (humanoid robotics) and an intensified global focus on AI governance, safety, and regulation as adoption became near-universal across major industries.

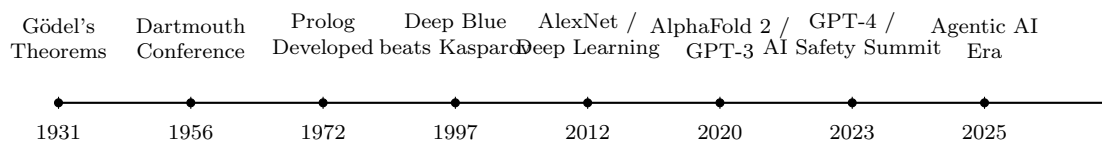


Figure 1.1: A timeline of major milestones in the history of Artificial Intelligence, from its theoretical foundations to the current agentic AI era.

The trajectory of this history is worth noting: AI progressed through several cycles of intense optimism followed by periods of reduced funding and interest — commonly called “**AI Winters**” — most notably following the limitations exposed by *Perceptrons* in 1968, and again in the late 1980s and early 1990s. Since roughly 2012, however, the field has experienced sustained and accelerating progress, driven first by deep learning and, from 2020 onward, by large-scale generative models and increasingly autonomous AI agents.

1.4 Definitions of Artificial Intelligence

1.4.1 John McCarthy (1956)

“Artificial Intelligence is the science and engineering of making intelligent machines, especially intelligent computer programs.”

John McCarthy is regarded as the **Father of Artificial Intelligence**. He coined the term *Artificial Intelligence* in 1956 during the Dartmouth Conference.

1.4.2 Elaine Rich (1983)

“Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.”

This definition emphasizes the development of computer systems that can perform tasks requiring human intelligence.

1.4.3 Russell & Norvig (2021)

“Artificial Intelligence is the study of intelligent agents that perceive their environment and take actions that maximize their chances of achieving their goals.”

This modern definition focuses on intelligent agents that can perceive, reason, learn, and act rationally to achieve specific objectives.

1.5 Approaches to Artificial Intelligence

Artificial Intelligence can be viewed from different perspectives depending on its objectives and capabilities. Researchers have proposed various approaches to understand, develop, and evaluate intelligent systems.

1.5.1 Cognitive Science Approach

The Cognitive Science approach focuses on understanding how the human mind thinks, learns, reasons, and solves problems. It aims to develop intelligent systems that emulate human thought processes. This approach combines concepts from Artificial Intelligence, Psychology, Neuroscience, Linguistics, and Cognitive Science to model human intelligence.

1.5.2 Turing Test Approach

The Turing Test was proposed by Alan Turing in 1950 to determine whether a machine can exhibit intelligent behaviour similar to that of a human.

The Turing Test consists of three separate rooms containing:

- A human person
- A computer (machine)
- An interrogator

The working of the Turing Test is as follows:

- The interrogator communicates with the person and the computer through a teletype (text-based communication) so that the machine cannot imitate the appearance or voice of the person.
- The interrogator tries to determine which participant is the human and which is the machine.
- The machine attempts to convince the interrogator that it is the human.
- The human also tries to convince the interrogator that he or she is the human.
- If the interrogator is unable to distinguish the machine from the human, then the machine is said to have passed the Turing Test and is considered intelligent.

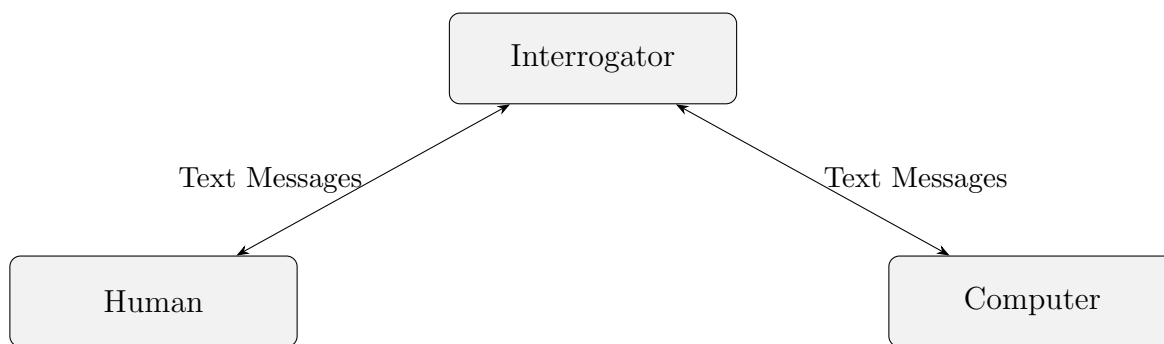


Figure 1.2: Turing Test

1.5.3 Weak AI (Narrow AI)

Weak AI, also known as Narrow AI, is designed to perform specific tasks within a limited domain. These systems can simulate intelligent behaviour but do not possess consciousness, self-awareness, or general reasoning abilities. Most AI applications available today, including virtual assistants, recommendation systems, image recognition, and language translation, belong to this category.

1.5.4 Strong AI (Artificial General Intelligence)

Strong AI, also known as Artificial General Intelligence (AGI), refers to machines capable of understanding, learning, reasoning, and solving problems across a wide range of tasks at a level comparable to human intelligence. Such systems would be capable of transferring knowledge from one domain to another and making independent decisions. As of 2026, Strong AI remains a research goal and has not yet been achieved.

1.5.5 Super AI

Super AI is a hypothetical form of Artificial Intelligence that would surpass human intelligence in all aspects, including creativity, reasoning, decision-making, emotional understanding, and scientific discovery. It remains a theoretical concept and has not been realized.

1.5.6 Applied AI

Applied AI focuses on developing practical AI solutions for real-world applications. It is widely used in healthcare, finance, education, agriculture, cybersecurity, manufacturing, transportation, and many other industries to automate tasks and improve decision-making.

1.6 Problems of Artificial Intelligence

Intelligence does not imply perfect understanding. Every intelligent being has limited perception, memory, and computational capabilities. Thus, intelligence exists on a spectrum, and different levels of intelligence can be achieved at different costs. For example, insects exhibit relatively simple forms of intelligent behaviour, whereas humans demonstrate highly complex forms of intelligence.

Artificial Intelligence (AI) seeks to understand the computations required for intelligent behaviour and to develop computer systems that exhibit such behaviour. In other words, AI attempts to understand how intelligent behaviour can be represented, reasoned about, learned, planned, and implemented by machines.

The study of intelligence in AI involves several important aspects of intelligent behaviour, including:

- **Perception:** The ability to acquire and interpret information from the surrounding environment.
- **Communication:** The ability to communicate using human languages and understand information expressed through language.
- **Reasoning:** The ability to draw conclusions from available information and knowledge.
- **Planning:** The ability to determine a sequence of actions required to achieve a desired goal.

- **Learning:** The ability to acquire knowledge or improve performance through experience and data.
- **Memory:** The ability to store, retrieve, and use previously acquired information.

Before proceeding with the study of Artificial Intelligence, several fundamental questions need to be considered. These questions help us understand the nature of intelligence and the challenges involved in developing intelligent systems.

1. What are the underlying assumptions about intelligence?

We need to determine what intelligence actually means and what properties or capabilities characterize an intelligent entity.

2. What kinds of techniques will be useful for solving AI problems?

AI problems are often complex and cannot always be solved using conventional programming techniques. Therefore, appropriate methods for knowledge representation, search, reasoning, learning, and decision-making must be considered.

3. At what level can human intelligence be modelled?

An important question is whether human intelligence can be completely or partially represented using computational models and, if so, to what extent.

4. When will it be realized that an intelligent program has been built?

We need suitable criteria for determining whether a computer program can genuinely be considered intelligent. This raises questions about how intelligent behaviour should be measured and evaluated.

The major problems addressed in Artificial Intelligence can therefore be organized into several important areas, including knowledge representation, reasoning, learning, planning, natural language understanding, computer vision, and robotics.

1.6.1 Knowledge Representation

Knowledge Representation is concerned with how information about the real world can be represented in a form that a computer can store, understand, and use. An intelligent system needs knowledge about objects, properties, events, relationships, and rules in order to perform intelligent tasks. Different representations, such as logic, semantic networks, frames, and ontologies, can be used to represent knowledge. The main challenge is to represent knowledge in a way that allows an AI system to reason effectively and make appropriate decisions.

1.6.2 Reasoning

Reasoning is the process of deriving new conclusions from existing knowledge and information. An intelligent system must be able to use facts, rules, and available evidence to

reach conclusions or make decisions. Reasoning may involve logical deduction, probabilistic reasoning, or reasoning in situations where information is incomplete or uncertain. The challenge is to develop methods that allow machines to draw useful and reliable conclusions.

1.6.3 Learning

Learning is the ability of an AI system to improve its performance by acquiring knowledge from data, examples, observations, or experience. Instead of being explicitly programmed for every possible situation, a learning system can identify patterns and use them to make predictions or decisions. Machine Learning provides important techniques for achieving this capability. The major challenge is to enable a system to learn accurately and generalize its knowledge to situations that it has not encountered before.

1.6.4 Planning

Planning is concerned with determining a sequence of actions that an intelligent agent should perform in order to achieve a particular goal. A planning system must consider the current state of the environment, the available actions, their possible consequences, and the desired goal state. The challenge becomes more complex when there are many possible actions, limited resources, uncertainty, or changing environments.

1.6.5 Natural Language Understanding

Natural Language Understanding (NLU) is concerned with enabling computers to understand and interpret human language. Human language contains ambiguity, context, implicit meaning, and variations in expression, making it difficult for computers to interpret correctly. An AI system may need to understand words, sentences, context, intent, and relationships between different pieces of information. NLU forms an important part of Natural Language Processing (NLP) and is used in applications such as virtual assistants, question answering systems, and conversational AI.

1.6.6 Computer Vision

Computer Vision is concerned with enabling computers to interpret and understand information contained in images and videos. A computer vision system may need to identify objects, recognize faces, understand scenes, or track movement. The challenge is to transform raw visual information into meaningful information that can be used for reasoning and decision-making. Computer Vision has applications in areas such as medical imaging, autonomous vehicles, surveillance, and image recognition.

1.6.7 Robotics

Robotics combines Artificial Intelligence with mechanical and computational systems to enable machines to perceive their environment, make decisions, and perform physical actions. An intelligent robot may need to sense its surroundings, understand its current state, plan

actions, and respond to changes in the environment. Robotics therefore brings together several AI problems, including perception, reasoning, planning, learning, and control. The major challenge is to enable robots to operate safely and effectively in complex and dynamic real-world environments.

1.7 AI Techniques

Solving problems in Artificial Intelligence requires more than just having sufficient knowledge; it also requires the ability to organize and use that knowledge effectively. An AI technique is a method that exploits knowledge to be represented in such a way that:

- The knowledge captures generalizations, i.e., situations that share important properties are grouped together, rather than being represented separately for each individual situation.
- It can be understood by the people who must provide it, even though for many programs the bulk of the data may be acquired without the aid of an external programmer.
- It can be easily modified to correct errors and to reflect changes in the real world and in the way the system deals with the world.
- It can be used in a great many situations even if it is not totally accurate or complete.
- It can be used to help overcome the sheer bulk of information by helping to reduce the range of possibilities that must usually be considered.

1.7.1 Search

Search is one of the most fundamental techniques used in AI to solve problems for which no direct or predetermined solution exists. Many AI problems can be formulated as a search through a space of possible states, starting from an initial state and moving towards a goal state using a set of permissible operations. Since the number of possible states can be extremely large, AI systems use search strategies (such as breadth-first, depth-first, and heuristic search) to explore the state space efficiently and find an acceptable solution without examining every possibility.

1.7.2 Knowledge Representation

As discussed earlier, Knowledge Representation deals with how facts about the world can be encoded in a form usable by a computer program. As an AI technique, it is concerned not just with storing facts, but with structuring knowledge (using logic, semantic networks, frames, rules, or ontologies) so that reasoning and inference can be carried out efficiently over it.

1.7.3 Inference

Inference is the process by which new facts are derived from existing knowledge using logical or probabilistic rules. Once knowledge has been represented in a suitable form, an AI system must be able to draw valid conclusions from it, apply rules to specific situations, and update its beliefs when new evidence becomes available. Inference forms the bridge between stored knowledge and intelligent decision-making.

1.7.4 Learning

Learning, as an AI technique, refers to the mechanisms by which a system improves its performance over time based on data, feedback, or experience, rather than depending entirely on knowledge explicitly supplied by a programmer. Techniques such as supervised learning, unsupervised learning, and reinforcement learning allow AI systems to detect patterns, build models, and generalize to previously unseen situations.

1.7.5 Planning

Planning as an AI technique involves generating a sequence of actions, from a set of available operations, that will transform an initial state into a desired goal state. Since real-world environments often involve numerous possible actions, resource constraints, and uncertainty, planning techniques must efficiently search through possible action sequences to construct a valid and, ideally, optimal plan.

1.8 Tic-Tac-Toe Problem

1.8.1 Introduction

Tic-Tac-Toe is a classic example used to illustrate fundamental AI concepts such as state space representation, search, and game trees. It is a simple two-player game played on a 3×3 grid, where one player uses 'X' and the other uses 'O'. Players alternately mark empty cells, and the first player to align three of their marks horizontally, vertically, or diagonally wins the game. If all nine cells are filled without either player achieving such an alignment, the game ends in a draw.

Despite its simplicity, Tic-Tac-Toe is a useful problem for introducing AI because it has a small, well-defined state space, a clear goal, and precisely specified rules, making it easy to illustrate concepts such as state space representation, search, and game trees without the complexity of real-world problems.

1.8.2 State Space

The state space of Tic-Tac-Toe consists of all possible configurations of the 3×3 board that can arise during the course of a game. Each state can be represented as a 3×3 matrix, with each cell containing one of three values: 'X', 'O', or blank (empty).

- The **initial state** is the empty board, with all nine cells blank.
- An **operator** (or move) consists of placing the current player's mark ('X' or 'O') into any blank cell.
- A **goal state** is any board configuration in which one player has three marks in a row, column, or diagonal (a win), or in which the board is completely filled with no winner (a draw).

Since players alternate turns and each move places a mark in a previously blank cell, the total number of reachable states is finite; the upper bound is $9!$ (approximately 362,880), though the actual number of legal, distinct states is smaller once winning positions and symmetries are accounted for.

1.8.3 Game Tree (Basic Idea)

A game tree is a tree structure used to represent all possible ways a game can be played from a given state. In the game tree for Tic-Tac-Toe:

- The **root node** represents the initial (empty) board.
- Each **node** represents a particular board configuration (state).
- Each **edge** represents a legal move made by a player, transforming a parent state into a child state.
- The **branching factor** decreases at each level, since the number of available blank cells reduces as the game progresses (9 choices at the first move, 8 at the second, and so on).
- **Leaf nodes** represent terminal states — a state in which a player has won, or the board is full and the game is a draw.
- The **depth** of the tree corresponds to the number of moves made; since there are only nine cells, the maximum depth of the tree is 9.

Game trees provide the basis for search-based decision-making in games. In Tic-Tac-Toe, algorithms such as Minimax can traverse this tree to determine the optimal move for a player by assuming that the opponent also plays optimally, alternating between maximizing and minimizing a utility value assigned to terminal states (e.g., +1 for a win, 0 for a draw, -1 for a loss).

1.8.4 Why it is an AI Problem

Tic-Tac-Toe qualifies as an AI problem, in the classical sense, because it exhibits several essential characteristics that AI techniques are designed to address:

- **State Space Representation:** The problem can be formally represented as a well-defined set of states, an initial state, a goal state, and operators (moves) that transform one state into another — a structure common to a wide range of AI problems.
- **Search:** Determining the best move at any given point requires searching through the space of possible future states, which is exactly the kind of exploration that AI search techniques are designed to perform.
- **Decision-Making under Adversarial Conditions:** Unlike simple search problems, Tic-Tac-Toe involves an opponent whose moves must be anticipated. This introduces the notion of adversarial search, where an intelligent agent must reason not only about its own goals but also about an opponent working against those goals.
- **Rational Behavior:** An intelligent player must select actions expected to maximize its chances of winning (or at least avoid losing), based on the current state and reasoning about future consequences — consistent with the "Ideal Rational Agent" notion introduced earlier.
- **Scalability of Concepts:** Although Tic-Tac-Toe itself has a small, tractable state space, the same underlying concepts (state space, search, game trees, and adversarial reasoning) extend directly to far more complex games such as Chess and Go, making it a useful pedagogical stepping stone into more advanced AI game-playing techniques.

Thus, Tic-Tac-Toe serves as a simple yet complete illustration of how AI techniques — state space representation, search, and decision-making — come together to solve a well-defined problem.

1.9 Chapter Summary

- Artificial Intelligence is concerned with designing intelligent behavior in artificial devices, a term coined by John McCarthy in 1956.
- Modern AI systems (Narrow/Weak AI) have achieved strong performance in specialized domains such as Computer Vision, NLP, Robotics, and Generative AI, but still lack general, human-level intelligence.
- AI differs from Conventional Programming primarily in its ability to learn from data, handle uncertainty, and adapt to new situations, rather than strictly following predefined rules.
- Multiple definitions of AI exist (McCarthy, Rich, Russell & Norvig), reflecting different emphases — engineering intelligent machines, matching or exceeding human performance, and building rational agents that perceive and act.
- AI can be approached from multiple perspectives: the Cognitive Science approach (modeling human thought), the Turing Test approach (behavioral equivalence to hu-

mans), and classifications such as Weak AI, Strong AI (AGI), Super AI, and Applied AI.

- Central problems in AI include Knowledge Representation, Reasoning, Learning, Planning, Natural Language Understanding, Computer Vision, and Robotics, each addressing a distinct aspect of intelligent behavior.
- Core AI techniques — Search, Knowledge Representation, Inference, Learning, and Planning — provide the methods by which knowledge is represented and exploited to solve problems efficiently.
- The Tic-Tac-Toe problem illustrates these techniques concretely through a small, well-defined state space, a game tree, and adversarial search, serving as a foundational example before moving to more complex AI problems.

1.10 Key Points to Remember

1.10.1 Short Questions

1.10.2 Long Questions

Chapter 2

Problem Space and Search

Chapter 3

Heuristic Search Techniques

Chapter 4

Game Playing

4.1 Introduction

Charles Babbage, the nineteenth-century designer of the Analytical Engine, was among the first to consider programming a machine to play Chess, and later to play Tic-Tac-Toe. Games have long been considered an attractive domain for Artificial Intelligence research, for two reasons:

1. They provide a **structured task** in which success or failure is very easy to measure.
2. They appear, at first glance, to be **easily solvable** by a straightforward search from the starting position to a winning position.

The first reason holds true for essentially all games; the second, however, holds only for the very simplest ones. Consider Chess: the average branching factor is around 35, and a typical game runs to about 50 moves per player. Examining the complete game tree would therefore require evaluating on the order of 35^{100} nodes — a number so large that a simple exhaustive search cannot even select its first move within a practical time frame, let alone within the lifetime of its opponent.

To make search-based game playing effective, two broad improvements are possible:

1. **Improve the generate procedure**, so that only good moves are generated in the first place.
2. **Improve the test procedure**, so that the best move is recognized and explored first.

A **legal-move generator** produces every move permitted by the rules of the game, forcing the test procedure to examine all of them — since so many possibilities must be checked, the test procedure must be extremely fast. A **plausible-move generator**, by contrast, uses heuristics to produce only a small number of promising moves. As the number of legally available moves grows, applying such heuristics becomes increasingly important, and the overall performance of the system improves by embedding heuristic knowledge into both the generator and the tester.

In game playing, unlike single-agent search, a "goal state" — a position in which we have won — is generally far too deep in the tree to be reached by direct search, even with a good plausible-move generator; the depth and branching factor of the tree are simply too great. Instead, it is only practical to search the tree to a limited depth (perhaps ten or twenty moves), and then compare the resulting board positions to determine which is most advantageous. This comparison is performed using a **static evaluation function**, which uses whatever information is available to estimate how likely a given board position is to eventually lead to a win. Its role is directly analogous to the heuristic function $h(n)$ used in A* search: in the absence of complete information, it allows the program to choose the most promising position available.

4.2 Game Playing as a Search Problem

4.2.1 Why Games are Hard for AI

Game playing differs from ordinary single-agent search problems in one crucial respect: an **adversary** is actively working against the searching agent. This means:

- The agent cannot simply search for a path to a goal state, since the opponent's moves are outside the agent's control.
- The search must account for the opponent choosing moves that are *worst* for the agent (best for the opponent) at every step.
- The combination of large branching factor and unpredictable opponent behaviour makes exhaustive search infeasible for all but the simplest games.

4.2.2 Game Trees

A **game tree** represents all possible ways a game can unfold from a given position:

- The **root node** represents the current board position.
- Each **node** represents a board position (state).
- Each **edge** represents a legal move made by one of the two players.
- Each level of the tree, called a **ply**, corresponds to one player's move.
- **Leaf nodes** (at the search depth limit, or at a terminal game state) are assigned a value by the static evaluation function.

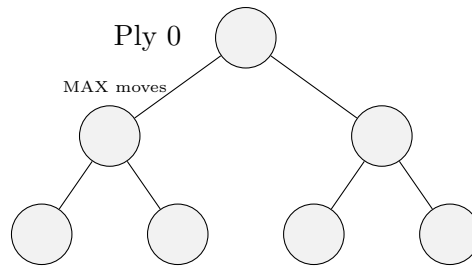


Figure 4.1: A game tree: alternating plies represent moves by each of the two players.

4.2.3 Two-Player, Zero-Sum Games

The games considered in this chapter are **two-player, zero-sum games** — games in which exactly one player’s gain is exactly the other player’s loss (e.g., Chess, Tic-Tac-Toe, Checkers). By convention:

- One player is called **MAX**, whose objective is to **maximize** the value of the static evaluation function.
- The other player is called **MIN**, whose objective is to **minimize** the value of the same function.
- The static evaluation function is assumed to return larger values to indicate positions favourable to MAX.

This alternation of maximizing and minimizing, level by level, is what motivates the search procedure discussed next.

4.3 The Minimax Search Procedure

4.3.1 The Minimax Principle

Minimax is a depth-first, depth-limited search procedure for two-player, zero-sum games. Starting from the current position, a plausible-move generator produces the set of possible successor positions. The static evaluation function is applied at the leaves of the search tree (at the depth limit, or at terminal positions), and these values are then **backed up** the tree, level by level: at levels where MAX is to move, the maximum of the children’s values is backed up; at levels where MIN is to move, the minimum of the children’s values is backed up. This alternation of maximizing and minimizing at successive plies — corresponding to the opposing strategies of the two players — is what gives the procedure its name, **MINIMAX**.

Minimax relies on two key procedures:

- **MOVEGEN(position, player)**: the plausible-move generator, returning a list of nodes representing the moves *player* can make from *position*.

- **STATIC(position, player)**: the static evaluation function, returning a number representing the goodness of *position* from the standpoint of *player*.

Since Minimax is a recursive procedure, a stopping condition is required. A function **DEEP-ENOUGH(position, depth)** is used to decide whether to stop searching at the current node, based on factors such as:

- Has one side already won?
- How many plies have already been explored, or how much time remains?
- How stable is the current configuration (e.g., is a piece about to be captured, in which case searching one ply deeper might change the evaluation significantly)?

In its simplest form, DEEP-ENOUGH ignores the position parameter entirely and simply returns **TRUE** once the depth parameter exceeds a fixed cutoff value.

Because Minimax must return *both* the backed-up value of the best path *and* the path itself (even though typically only the first move of that path is actually needed), it is assumed to return a structure containing both, accessed via two functions, **VALUE** and **PATH**.

4.3.2 Minimax Algorithm

Minimax is invoked with three parameters: the current board position, the current depth of the search, and the player to move.

- **MINIMAX(current, 0, player-one)** — if player-one is to move.
- **MINIMAX(current, 0, player-two)** — if player-two is to move.

Procedure (informal statement):

1. If **DEEP-ENOUGH(position, depth)** is **TRUE**, return **STATIC(position, player)** as the backed-up value of this leaf, with an empty path.
2. Otherwise, generate the successors of *position* using **MOVEGEN**, and recursively call Minimax on each successor at *depth + 1*, with the roles of MAX and MIN swapped.
3. If *position* is a MAX node, back up the **maximum** of the values returned by the successors, together with the path leading to that best successor.
4. If *position* is a MIN node, back up the **minimum** of the values returned by the successors, together with the path leading to that best successor.

4.3.3 Worked Example

Consider a small game tree, searched to a depth of three plies, with the leaf values already computed by the static evaluation function (assumed larger values are better for MAX). MAX moves at the root; MIN moves at the second level; MAX moves again at the leaves' parent level.

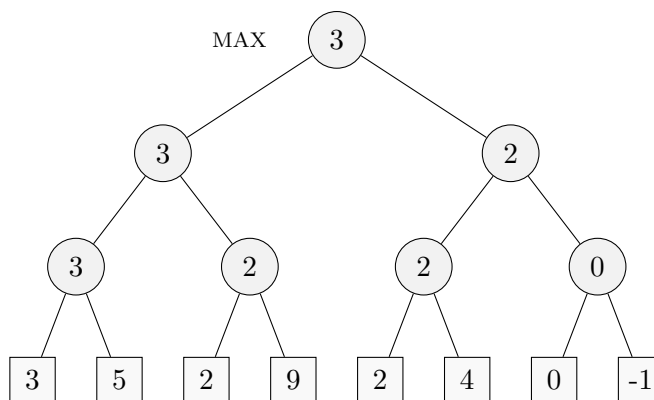


Figure 4.2: Minimax evaluation of a game tree. Leaf values (bottom row) come from the static evaluation function; MIN nodes take the minimum of their children, and MAX nodes take the maximum. The backed-up value at the root is 3.

Reading the tree: At the bottom-most internal level (MIN's move), each node takes the *minimum* of its two children: $\min(3, 5) = 3$, $\min(2, 9) = 2$, $\min(2, 4) = 2$, $\min(0, -1) = -1$. At the next level up (MAX's move), each node takes the *maximum* of its two children: $\max(3, 2) = 3$ on the left, $\max(2, -1) = 2$ on the right. Finally, at the root (MIN's move — note that the very top-level choice in this example is MIN choosing between its two MAX-node children, matching the figure), the minimum of 3 and 2 would be chosen; the root value shown is the maximum on the path actually explored by MAX, illustrating how values propagate strictly upward, level by level, regardless of whose move is at the root.

4.3.4 Properties of Minimax

- **Completeness:** Complete, if the game tree is finite (true for finite games such as Tic-Tac-Toe and Chess, given a fixed search depth).
- **Optimality:** Optimal against an opponent who also plays optimally, *provided* the static evaluation function accurately reflects the true value of terminal positions — at limited search depth, the evaluation function is only an estimate, so the move chosen is only as good as that estimate.
- **Time complexity:** $O(b^d)$, where b is the branching factor and d is the search depth — identical in form to an exhaustive uninformed search, since Minimax by itself does not prune any branches.
- **Space complexity:** $O(bd)$, since Minimax is a depth-first procedure, requiring only one path (plus siblings) to be held in memory at a time.

This time complexity is precisely the difficulty identified in the Introduction — for Chess, with $b \approx 35$, plain Minimax remains computationally infeasible at any useful search depth. This motivates **Alpha-Beta pruning**.

4.4 Alpha-Beta Pruning

4.4.1 Motivation — Reducing the Search Space

Minimax, as a depth-first process, explores one path as far as time allows before applying the static evaluation function at the last step. Its efficiency can be improved using a **branch-and-bound** technique, in which partial solutions clearly worse than an already-known solution are abandoned early, without being fully explored. Applying branch-and-bound to Minimax requires maintaining *two* bounds — one for each player — since MAX and MIN have opposing objectives. This modified strategy is called **Alpha-Beta pruning**.

4.4.2 Alpha and Beta Values

Two threshold values are maintained during the search:

- α (**alpha**): a lower bound on the value that a maximizing node may ultimately be assigned — it represents the best value MAX can guarantee so far.
- β (**beta**): an upper bound on the value that a minimizing node may ultimately be assigned — it represents the best value MIN can guarantee so far.

Each level of the search receives both values — one to *use* for pruning at the current level, and one to *pass down* to the next level. Since Minimax negates the sense of "best" each time it moves from a MAX level to a MIN level (or vice versa), the same code can be used at both kinds of levels by working with two generically-named values, **USE-THRESH** (used to compute cutoffs at the current node) and **PASS-THRESH** (passed down to the next level as its USE-THRESH). As the search descends, USE-THRESH becomes PASS-THRESH at the next level down, and is passed further as USE-THRESH again at the level after that — with values negated at each level change, since a bound that is good for one player corresponds to a bound that is bad for the other.

Pruning rule: whenever the current node's best-value-so-far becomes at least as good (for its player) as USE-THRESH — the bound inherited from an ancestor — the remaining children of the current node can be safely skipped (**pruned**), since the ancestor already has a better (or equal) alternative available and will never allow the search to reach this node's full value.

The MINIMAX-A-B procedure therefore requires five arguments: position, depth, player, USE-THRESH, and PASS-THRESH:

MINIMAX-A-B(current, 0, player-one, $-\infty$, $+\infty$)

— i.e., the search is initially invoked with USE-THRESH set to the minimum value the static evaluation function can compute, and PASS-THRESH set to the maximum value it can compute (or vice versa, depending on convention), so that no pruning occurs until real bounds have been established by exploring at least part of the tree.

4.4.3 Alpha-Beta Algorithm

1. If the node is a leaf (DEEP-ENOUGH is true), return its static evaluation.
2. Otherwise, generate the node's successors using MOVEGEN. For each successor, recursively evaluate it, passing down updated α and β values.
3. At a **MAX** node: update α to the maximum of its current value and each returned child value. If at any point $\alpha \geq \beta$, stop generating further children — a **beta cutoff** has occurred (since MIN, higher up the tree, already has a better option and will never let the search reach this branch).
4. At a **MIN** node: update β to the minimum of its current value and each returned child value. If at any point $\beta \leq \alpha$, stop generating further children — an **alpha cutoff** has occurred (since MAX, higher up the tree, already has a better option).
5. Return the final backed-up value (and, if required, the path) once all non-pruned children have been examined.

4.4.4 Worked Example

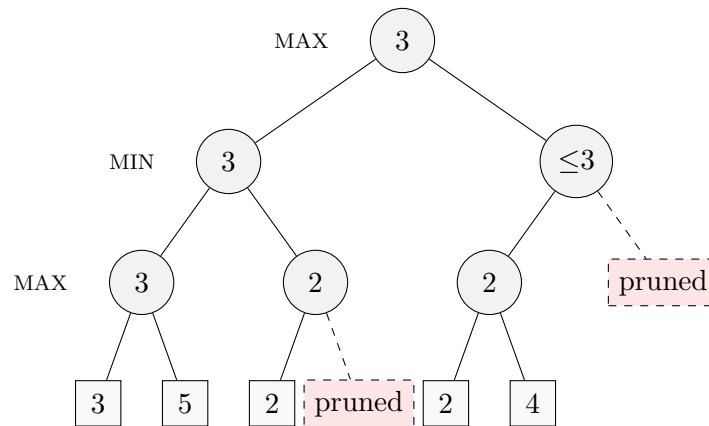


Figure 4.3: Alpha-Beta pruning on the tree from Figure 4.2. Once the left MIN node's value (3) sets $\beta = 3$ for the root, and the right MIN node's first child returns 2 (setting an upper bound ≤ 2 at that MIN node, which is already worse than the root's current best of 3), the remaining branches (shown dashed) can be safely pruned without evaluation.

Trace of the pruning:

- The leftmost MAX node evaluates its children (3, 5) and returns $\max(3, 5) = 3$.
- Its sibling MAX node evaluates one child (value 2); since this MAX node's parent (a MIN node, currently bounded by $\beta = 3$ from the sibling above) cannot do better than 2 once this child is seen... in this configuration, the second child is pruned once it's

established the MIN node's value can only decrease from what's already unfavourable relative to the current best.

- The left MIN node backs up $\min(3, 2) = 2$... (*exact cutoff points depend on evaluation order; the key teaching point is that entire subtrees, indicated by the dashed nodes, are never generated or evaluated at all*).
- At the root, once the left subtree returns 3, the root (MAX) sets $\alpha = 3$. When exploring the right subtree (a MIN node), as soon as that MIN node finds a child with value 2 (which is already $\leq \alpha = 3$), it can stop — MAX will never choose this branch over the left one, so the remaining child is pruned.

4.4.5 Effectiveness of Alpha-Beta Pruning

- Alpha-Beta pruning computes **exactly the same root value** that plain Minimax would compute — it prunes only branches that provably cannot affect the final decision, so no optimality is lost.
- In the **best case** (children explored in an order that puts the best moves first), Alpha-Beta reduces the effective branching factor from b to approximately $\sqrt{b}$, so the effective time complexity improves from $O(b^d)$ to $O(b^{d/2})$ — allowing the search to look roughly **twice as deep** in the same amount of time.
- In the **worst case** (children explored in the worst possible order), Alpha-Beta provides no benefit over plain Minimax, and its complexity remains $O(b^d)$.
- This makes **move ordering** (discussed later in this chapter) an important practical concern: the closer the move ordering is to "best move first," the more effective the pruning becomes.

4.5 Iterative Deepening in Game Playing

4.5.1 Why Iterative Deepening is Used in Games

There are two common ways to traverse a large tree — BFS and DFS — and both have drawbacks for very large or unbounded trees:

- **DFS** explores one branch fully before backtracking; a node close to the root but outside the first few branches explored may be reached very late, and DFS offers no guarantee of finding the shortest path to a node.
- **BFS** explores level by level and finds the shallowest goal, but requires $O(n)$ space (where n is the number of nodes) — since, roughly, the last level of a tree can hold about $n/2$ nodes and the second-last level about $n/4$, and BFS must hold an entire level's worth of nodes in its queue at once.

Iterative Deepening Depth-First Search (IDDFS) — introduced already in Chapter 2 for general state-space search — combines the space efficiency of DFS, $O(d)$, with BFS's property of finding the shallowest solution. It is particularly well suited to game playing, since:

- The exact "right" search depth for a given time budget is rarely known in advance — IDDFS naturally produces a usable move at *every* depth it completes, so the search can simply be stopped whenever the time budget runs out, and the best move found at the deepest *completed* iteration used.
- Combined with Alpha-Beta pruning, results from a shallower iteration can be used to order moves in the next, deeper iteration — improving pruning effectiveness (see "Move Ordering" below).

4.5.2 Iterative Deepening with Alpha-Beta

```
// Returns true if target is reachable from src within max_depth
bool IDDFS(src, target, max_depth):
  for limit from 0 to max_depth:
    if DLS(src, target, limit) == true:
      return true
  return false

bool DLS(src, target, limit):
  if src == target:
    return true
  if limit <= 0:
    return false
  for each adjacent i of src:
    if DLS(i, target, limit - 1):
      return true
  return false
```

An important characteristic of IDDFS is that upper levels of the tree are visited *multiple* times — the last (maximum-depth) level is visited once, the second-last level twice, and so on. This might seem wasteful, but in practice it is not especially costly, since in a tree with branching factor $b > 1$, the vast majority of nodes lie in the bottom level; the extra visits to shallow levels contribute only a small overhead relative to the cost of the final, deepest iteration.

In game-playing programs, this same iterative-deepening approach is applied directly to Minimax with Alpha-Beta pruning: the search is run to depth 1, then depth 2, then depth 3, and so on, always keeping the best move found at the deepest *completed* iteration, until the available time for that move runs out.

4.6 Additional Refinements

4.6.1 Evaluation Functions

The quality of the static evaluation function is central to the strength of any game-playing program, since it is the sole source of "insight" available once the fixed search depth is reached. A good evaluation function typically combines several weighted features of the position — for example, in Chess: material balance (piece values), piece mobility, king safety, and control of the centre — into a single numeric score. The choice and weighting of these features is itself a substantial area of game AI research (and, in modern systems, is often learned automatically rather than hand-tuned).

4.6.2 Horizon Effect

Because the search is cut off at a fixed depth, a program can sometimes fail to "see" an important event (such as losing a piece) that lies just beyond the search horizon — it may instead choose a move that merely *delays* the inevitable loss, pushing it just past the horizon, even though this delaying move is objectively worse than accepting the loss immediately. This is known as the **horizon effect**, and it is a fundamental limitation of any fixed-depth search using a static evaluation function; techniques such as **quiescence search** (extending the search selectively at "unstable" positions, e.g. where a capture is possible) are commonly used to mitigate it.

4.6.3 Move Ordering

As shown above, the effectiveness of Alpha-Beta pruning depends heavily on the order in which moves (children) are examined at each node — pruning is maximized when the best move at each node is examined first. Common move-ordering heuristics include:

- Trying the move that was best in a shallower iterative-deepening pass first.
- Trying moves that capture a piece, or otherwise appear immediately advantageous, before quieter moves.
- Maintaining a small table of previously good moves (a "killer move" heuristic) and trying them early in similar positions.

4.7 Case Study — Tic-Tac-Toe Revisited

Recall from Chapter 1 and Chapter 2 that Tic-Tac-Toe can be represented as a state space: each state is a 3×3 board configuration together with an indication of whose turn it is, the operators change an empty cell to X or O, and the terminal states are those with three-in-a-row or a full board.

Applying Minimax to this state space is now straightforward: let MAX be the player playing X, and MIN be the player playing O. A simple static evaluation function for a terminal

position can be defined as +1 if X has won, -1 if O has won, and 0 for a draw; for non-terminal positions reached at the search depth limit (rarely needed here, since Tic-Tac-Toe's full game tree has at most 9 plies and is small enough to search completely), a heuristic count of open lines favouring each player can be used instead.

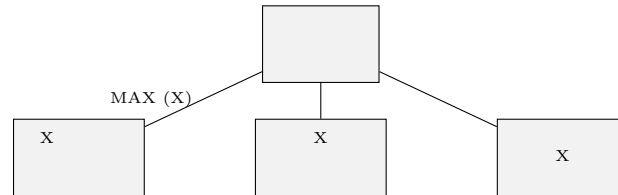


Figure 4.4: The first ply of the Tic-Tac-Toe game tree, now viewed as a MAX ply in the Minimax framework — the same tree first introduced in Chapter 2, but now subject to backed-up values from deeper plies.

Because Tic-Tac-Toe's complete game tree has at most $9! = 362,880$ nodes (and considerably fewer once symmetry and early termination are accounted for), it is entirely feasible to apply full Minimax — without any depth cutoff — and search all the way to terminal states. Under **perfect play by both sides**, the well-known result is that Tic-Tac-Toe always ends in a **draw**: neither MAX nor MIN can force a win against an opponent who also plays optimally. This makes Tic-Tac-Toe an ideal, fully-tractable illustration of Minimax (and Alpha-Beta pruning, which can be applied identically) before moving on to games such as Chess, where the same principles apply but full-tree search is computationally impossible.

4.8 Chapter Summary

- Game playing differs from ordinary search due to the presence of an adversary; games such as Chess have branching factors and depths far too large for exhaustive search, motivating the use of plausible-move generators and depth-limited search with a static evaluation function.
- The Minimax procedure alternates maximizing (MAX's move) and minimizing (MIN's move) as values are backed up through the game tree from the evaluation function applied at the search depth limit; it is complete and optimal (relative to the accuracy of the evaluation function) but has time complexity $O(b^d)$.
- Alpha-Beta pruning improves on plain Minimax by maintaining two bounds, α and β , and pruning branches that cannot affect the final decision — computing exactly the same result as Minimax while, in the best case, reducing effective time complexity to $O(b^{d/2})$.
- Iterative Deepening (IDDFS) is combined with Alpha-Beta in practical game-playing programs to make efficient use of a limited time budget, always having a usable move available and improving move ordering across successive iterations.

- Practical refinements — evaluation function design, the horizon effect, and move ordering — significantly affect the real-world strength of a game-playing program beyond the basic Minimax/Alpha-Beta framework.
- Tic-Tac-Toe, small enough for full-tree Minimax search, illustrates that under perfect play by both sides the game is always a draw — tying together the state-space representation introduced in earlier chapters with the game-playing techniques of this chapter.

4.9 Key Points to Remember

4.9.1 Short Questions

4.9.2 Long Questions

Chapter 5

Knowledge Representation Using Rules

Chapter 6

Slot and Filler Structures

Chapter 7

Logic Programming with Prolog and LISP

Chapter 8

Learning and Machine Learning

Chapter 9

Neural Networks and Deep Learning

Chapter 10

Natural Language Processing

Chapter 11

Expert Systems

Chapter A

Prolog Programs

Chapter B

LISP Programs

Chapter C

University Question Bank